

Maintainability Index: A Practical Guide for C# Developers

June 1, 2026

9 minutes read



Maintainability Index: A Practical Guide for C# Developers



If you have ever opened a code analysis report and seen a score labelled “Maintainability Index” sitting next to one of your classes, you have probably wondered two things at once: where does that number come from, and should I actually care about it?

The short answer is that the Maintainability Index (MI) is a composite score, and one of the ingredients baked into it is a much older and stranger metric called the Halstead Volume. This article covers both metrics, shows the formulas, works through a real C# example by hand, and tells you when they are useful and where they might mislead you.

Index

Quick definitions

Halstead Volume measures the “size” of a piece of code in terms of information content. It counts how many operators and operands a method uses, both in total and as distinct symbols, then combines them. More vocabulary and more tokens mean more volume.

Maintainability Index is a 0-to-100 score that blends Halstead Volume, [Cyclomatic Complexity](#), and [Lines of Code](#) into a single estimate of how painful a method, class, or assembly will be to maintain. Higher is better. Microsoft’s tooling treats anything above 20 as “okay” on its rebased scale, while at NDepend, above 50 is OK and above 80 is great.

The Halstead Metrics

Maurice Halstead published his “software science” in 1977 with an ambitious goal: to treat source code as a measurable physical quantity, the way you would measure mass or volume. The idea was that any program is just a stream of two kinds of tokens.

Operators: things that do something. Keywords like `if` and `return`, arithmetic and logical operators such as `+` and `&&`, assignment, method calls, and grouping symbols.

Operands: things that get acted on. Variables, constants, and literal values.

From those two categories Halstead derived four raw counts:

Everything else is built on top of those four numbers:

Of all of these, Volume is the one that survived into mainstream tooling, because it is the term that feeds the Maintainability Index. Intuitively, Volume answers: “how many bits would I need to write this method out, given its vocabulary?” A method that reuses a handful of variables and operators has a small volume; a sprawling method that touches dozens of distinct symbols has a large one.

A worked Halstead example in C#

Here is a small, ordinary method to measure:

I will tokenise the body using a simple source-level convention (real tools differ in the edge cases, which is exactly the point – more on that later). Operators and how often each appears:

So we have $n1 = 5$ distinct operators and $N1 = 11$ total operator occurrences. Operands and their counts:

That gives $n2 = 7$ distinct operands and $N2 = 11$ total operand occurrences. Plugging in:

So the Halstead Volume of `IsLeapYear` is roughly 79. On its own that number means almost nothing. It only becomes interesting when you compare it to other methods, or when you feed it into the Maintainability Index. That is what we will do next.

The Maintainability Index

The Maintainability Index was introduced in 1992 by Paul Oman and Jack Hagemeister at the University of Idaho. They were trying to answer a practical question for Hewlett-Packard: given an unfamiliar codebase, can we predict how hard it will be to maintain before we sink engineers into it? They ran a regression against a set of C and Pascal systems and produced a now-famous formula combining Halstead Volume, McCabe's Cyclomatic Complexity, and Lines of Code.

The original formula could return large positive numbers and unbounded negative ones, which made it awkward to display. When Microsoft shipped code metrics in Visual Studio, they clamped and rescaled it to a 0-100 range. That rescaled version is the one nearly everyone uses today, and it is also the one NDepend reports:

where:

Notice what the formula is really saying. Three things drag the score down: a large vocabulary (Halstead Volume), lots of branching (Cyclomatic Complexity), and sheer length (Lines of Code). The two logarithms matter a great deal – they mean the penalty for growth is steep at first and then flattens. Going from 10 to 50 lines hurts far more, proportionally, than going from 500 to 540.

Finishing the worked example

Back to **IsLeapYear**. We already have HV = 78.9. We need the other two inputs.

Cyclomatic Complexity counts the independent paths. There are two **if** statements, each adding a branch, so CC = 3 (one baseline path plus two decisions). The **==** comparison in the final return produces a boolean but does not branch, so it does not count.

Lines of Code: treating each statement as a logical line, this method is about 5 logical lines.

Now substitute everything in:

So **IsLeapYear** scores a Maintainability Index of roughly 71.

How should you read 71? That depends entirely on whose scale you trust, and that is the most confusing part of the topic.

Reading the Maintainability Index score

This is where you have to be careful, because two reputable tools will look at the same code and label it differently – not because they compute the index differently, but because they draw the lines in different places.

Microsoft's Visual Studio uses a deliberately conservative banding on the 0-100 scale:

Score	Colour	Meaning
0 – 9	Red	Low maintainability
10 – 19	Yellow	Moderate maintainability
20 – 100	Green	Good maintainability

Yes, that means Visual Studio considers a score of 20 to be “good.” Microsoft chose those thresholds explicitly to keep noise down – they only wanted red to appear when they were highly confident there was a real problem. The result is that almost everything shows green, which is comforting and not very actionable.

A stricter interpretation lines up closer to the original research and to how NDepend frames it:

Score	Meaning
80 – 100	Easy to maintain
50 – 79	Moderate, keep an eye on it
0 – 49	Hard to maintain, consider refactoring

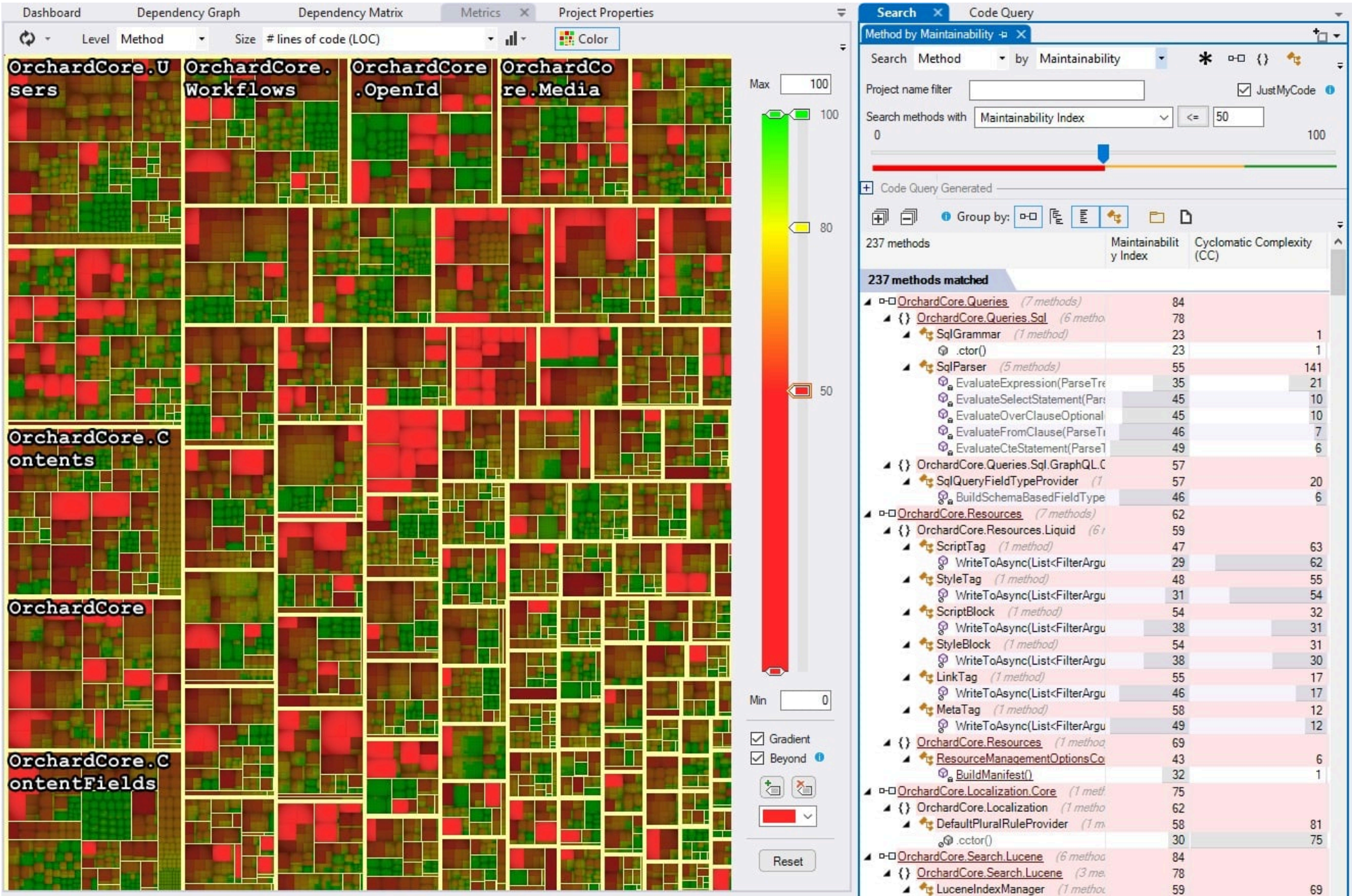
Under the Visual Studio bands, our **IsLeapYear** at 71 is comfortably green. Under the stricter interpretation it is merely “moderate” – which feels about right for a tiny method that frankly should score higher. That gap is your first hint that the absolute number is less trustworthy than the relative one.

Using these metrics well on a real C# codebase

Here is how to use Halstead Volume and the Maintainability Index day to day.

Rank, do not grade

Forget the absolute number. Sort your methods by Maintainability Index ascending and look at the bottom of the list. The worst 1% is where your refactoring time pays off, and relative comparison sidesteps the dodgy coefficients entirely. In NDepend, the Search by Maintainability tab does exactly this, and matches are highlighted in the metric view:



Combine, do not isolate

A low MI plus high Cyclomatic Complexity plus high coupling is a genuine hotspot. A low MI on its own might just be a long, dumb mapping method that nobody ever touches. Cross-reference with how often the code actually changes (your version control history knows) before you act. The code query below matches such methods.

```
from m in JustMyCode.Methods where
    m.MaintainabilityIndex < 50 &&
    m.CyclomaticComplexity > 10 &&
    m.MethodsCalled.Count() > 10
select new {
    m,
    m.MaintainabilityIndex,
    m.CyclomaticComplexity,
    m.MethodsCalled
}
```

Group by:	Maintainability Index	Cyclomatic Complexity	MethodsCalled
151 methods matched			
OrchardCore.Abstractions (1 method)	90		
OrchardCore.Environment.Shell.Configuration.Internal (1 method)	72		
ConfigurationExtensions (1 method)	63	18	
ToJToken(IConfiguration)	49	13	18 methods
OrchardCore.ResourceManagement.Abstractions (1 method)	89		
OrchardCore.ResourceManagement (1 method)	88		
ResourceDefinition (1 method)	77	116	
GetTagBuilder(RequireSettings,String,IFileVersio	33	41	47 methods
OrchardCore.Infrastructure.Abstractions (2 methods)	95		
System.IO (1 method)	61		
PathExtensions (1 method)	61	24	
ResolvePath(String)	45	16	21 methods
OrchardCore.Environment.Commands (1 method)	92		
DefaultCommandHandler (1 method)	63	41	
GetInvokeParametersForMethod(MethodInfo,IList	45	15	24 methods
OrchardCore (8 methods)	76		
OrchardCore.Modules (2 methods)	77		
ModularBackgroundService (2 methods)	56	65	
RunAsync(IEnumerable<ValueTuple<String,	44	16	65 methods
UpdateAsync(ValueTuple<String,Int64>[]V	46	20	63 methods
OrchardCore.Modules.Services (1 method)	64		
SlugService (1 method)	64	16	
Slugify(String)	49	12	16 methods
OrchardCore.Environment.Shell (1 method)	73		
ShellDescriptorFeaturesManager (1 method)	52	25	
UpdateFeaturesAsync(ShellDescriptor,IEnumeral	41	16	65 methods
OrchardCore.Environment.Shell.Removing (1 method)	62		
ShellRemovalManager (1 method)	51	17	
RemoveAsync(ShellSettings,Boolean)	44	16	62 methods

Watch the trend, not the snapshot

A single MI value tells you little. The same value drifting down release after release tells you a lot. Trend lines are where these metrics earn their keep. For example, here is a code query that matches methods whose Maintainability Index degraded since the baseline:

And here is a screenshot of these matches on the OrchardCore codebase:



Set a gate on regression, not on perfection

Failing a build because one method scored 64 is annoying and pointless. Failing it because a method that used to score 80 just dropped to 55 in this pull request is a conversation worth having. For example, here is a draft rule to detect such methods:

Frequently asked questions

What is a good Maintainability Index score?

On the rescaled 0-100 range, above 80 is comfortably maintainable, 50-79 is moderate and worth monitoring, and below 50 suggests the code is hard to maintain and may deserve refactoring. Note that Visual Studio uses a far more lenient banding where anything above 20 is shown as green.

Is a higher or lower Maintainability Index better?

Higher is better. The index runs from 0 (very hard to maintain) to 100 (easy to maintain).

What is the difference between Halstead Volume and the Maintainability Index?

Halstead Volume measures one thing – the information content of code based on its operators and operands. The Maintainability Index is a composite score that uses Halstead Volume as one of three inputs, alongside Cyclomatic Complexity and Lines of Code.

How is Halstead Volume calculated?

Volume is $V = N * \log_2(n)$, where N is the total number of operator and operand occurrences and n is the count of distinct operators and operands (the program's vocabulary).

Why does my tool report a different Maintainability Index than my hand calculation?

Most tools, including NDepend, compute Halstead Volume from compiled IL rather than from source text, and they may count logical lines of code differently than you do. Both choices change the inputs to the formula, so the final number shifts.

Should I use the Maintainability Index to fail my build?

Use it to catch regressions rather than to enforce an absolute floor. Failing a build when a method's score drops sharply within a single change is meaningful; failing because a method sits one point below an arbitrary threshold usually just generates noise.

Key takeaways

Halstead Volume and the Maintainability Index are decades old, lean heavily on code size, and were calibrated on languages that are not C#. Despite all that, they remain a fast, cheap way to point a flashlight at the parts of a codebase most likely to hurt. Use them to rank and to spot trends, pair them with Cyclomatic Complexity and real change history, and never mistake a single green number for a healthy design. The metric is the smoke detector. You still have to walk into the room.